

Chorex: Restartable, Language-Integrated Choreographies

Anonymous author

Anonymous affiliation

Anonymous author

Anonymous affiliation

Abstract

We built Chorex, a language that brings choreographic programming to Elixir as a path toward robust distributed applications. Chorex is unique among choreographic languages because it tolerates failure among actors. Actors can crash and Chorex can spawn a new process, restore state, and reconfigure the network topology via out-of-band messages to other actors. Chorex also proves that full-featured choreographies can be implemented via metaprogramming, and thereby achieve tight integration with the host language. For example, mismatches between a choreography and a local actor are reported statically and in terms of source code (rather than generated code). This paper illustrates Chorex on several examples, ranging from a higher-order bookseller to a secure remote password protocol, and details its design and implementation. We offer reasons that Chorex's projection strategy is a viable approach for other languages to support restartable actors.

2012 ACM Subject Classification Software and its engineering → Extensible languages

Keywords and phrases Choreographic programming, Metaprogramming, Macros

Digital Object Identifier 10.4230/LIPIcs...

Category Research

1 Introduction

Choreographic programming adds a layer of organization to concurrent (or distributed) systems. Traditionally, concurrent systems are designed with a *global* view of how communications ought to proceed in mind (represented, e.g., as a sequence diagram [41]), but implemented in an entirely different fashion — as a set of *local* actors spread throughout a software system. The code in different actors can easily fall out of sync because only end-to-end testing holds actors together. Choreographic programming manifests the global view as a program and automatically projects this *choreography* to local actor programs [7, 39, 40].

With choreographies, classes of communication errors become unrepresentable. Sends and receives cannot be mismatched because they are paired by design in a combined form, `send ~> recv`, in a choreography. Deadlocks cannot occur because lexical scope rules them out. For example, our language Chorex reports a compile-time error for the choreography below because the variable `A.val` is used before it is bound:

```
defmodule Deadlock do
  import Chorex
  defchor [A, B, C] do
    def run() do
      A.val ~> B.val
      B.val ~> C.val
      C.val ~> A.val
    end
  end
end
```

```
ERROR: undefined variable "val"
|
|           A.val ~> B.val
|           ~~~
|
+- deadlock.exs:6:10: A.run/1
```



© Anonymous author(s);

licensed under Creative Commons License CC-BY 4.0



Leibniz International Proceedings in Informatics

LIPIC Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

an implementation is notable because it follows the *languages as libraries* [52] method to achieve a high level of integration with its host language. Figure 1 presents two benefits of language integration:

- Figure 1a shows that functions required by a choreography appear as suggestions when a programmer edits an actor module. The pictured suggestion appears in the context of an actor module named `MyAlice` that fills the role named `:alice` from a choreography named `Demo.Chorex` (not pictured). There are two required functions: `decrypt` and `priv_key`.

- Figure 1b shows a tooltip box with a compile-time error due to missing annotations. This sort of error illustrated in Figure 1b is not detectable in choreographies implemented as runtime libraries because it requires two passes over the input [4, 47]. Either a bespoke compiler or metaprogramming in the style of Chorex is needed.

Both affordances are arguably IDE-agnostic because they leverage the Elixir language server. Our IDE of choice happens to be Emacs, but VSCode users could see similar (one might say “prettier”) tooltips.

Concretely, the Chorex compiler is an Elixir macro that analyzes source code, transforms the code to actor implementations, and propagates source locations so that errors in the output are manifest with actionable source locations. There are several other aspects of Chorex that, although not central to our research contributions, make it a promising tool for Elixir developers. Features include first-class functions, out-of-order message receives, message transport over TCP or basic sockets, and shared state across different instantiations of the same choreography (details in Section 4).

Outline

This paper begins with an example-driven tour of the Chorex language, starting with a description of a simple end-to-end choreography and culminating in a zero knowledge password proof (Section 2). These examples serve a dual purpose as an evaluation of Chorex, illustrating its expressiveness on realistic protocols. Next, the paper presents the design of Chorex (Section 3), with emphasis on its novel support for restartable actors, and follows with a close look at how Chorex achieves beneficial language integration through metaprogramming (Section 4). The paper concludes with a landscape of the rapidly-evolving area of choreographic programming (Section 6) and a discussion of next steps (Section 7).

Data Availability Statement

Chorex is open source and available on GitHub. We plan to submit an artifact that contains the latest release of Chorex and code that matches all examples in this paper.

2 Chorex By Example

Chorex is a domain-specific language for choreographic programming in Elixir. This section illustrates the basics of Chorex through several motivating examples.

Elixir [48, 50] is a compelling target language for several reasons. First, it compiles to the Beam VM (the Erlang virtual machine), and thus has access to primitives that support low-latency, distributed, fault-tolerant systems. These primitives have enabled rapid prototyping of choreography features. Second, Elixir has a large userbase that we hope to engage with in future work. Third, unlike its close relative Erlang, Elixir comes with a hygienic macro system. Thanks to macros, the Chorex language is implemented as a library and integrates smoothly with the Elixir build system, Mix [22], and package manager, Hex [24].

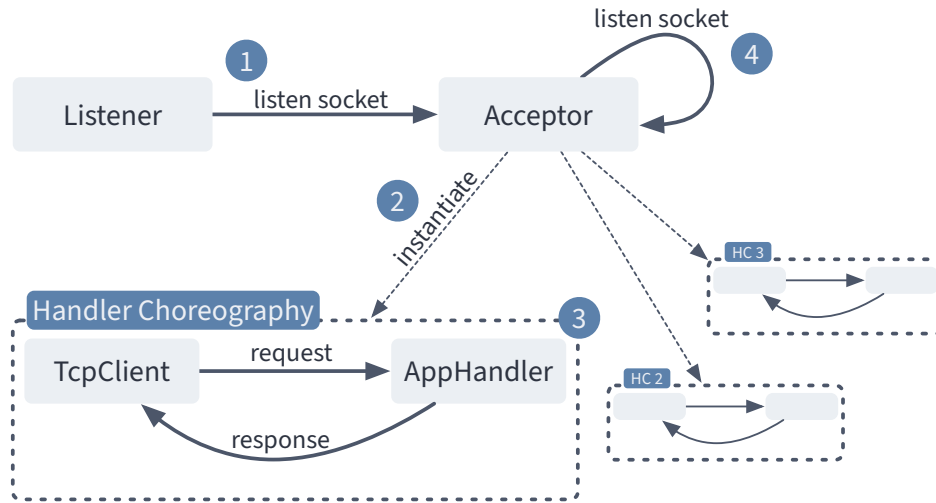


Figure 2 Socket server architecture: (1) Listener sends a socket to Acceptor; (2) Acceptor waits for a TCPClient, then spawns a Handler Choreography; (3) TCPClient and AppHandler exchange messages; (4) Acceptor loops and waits for additional TCPClients.

2.1 Socket Server

Our first example is a minimal socket server inspired by Thousand Island [51], a full-fledged socket server written in Elixir. There are two choreographies in our minimal server. A first choreography, between actors Listener and Acceptor, starts a server that clients can connect to. A second choreography specifies interactions between a Client and an AppHandler actor. Figure 2 maps out the overall design.

The key aspect of this example is that the first choreography can start arbitrarily many instances of the second choreography. Every client that connects to the server gets forwarded to a unique choreography with an app handler actor.

Each Chorex choreography must include a `run` function as an entry point, and may include additional functions (introduced by the `def` keyword). Our first choreography expects configuration data as input and immediately calls a Listener helper function to acquire a socket connection. The Listener sends this socket to an Acceptor actor, which calls the recursive function `loop`. Inside the loop, the Acceptor calls its own helper function to await clients. The Listener has nothing else to do in this simple choreography:

```
defmodule Tcp.ListenerChor do
  import Chorex

  defchor [Listener, Acceptor] do
    def run(Listener.(config)) do
      Listener.get_listener_socket(config) ~> Acceptor.({:ok, socket})
      loop(Acceptor.(socket))
    end

    def loop(Acceptor.(listen_socket)) do
      Acceptor.accept_and_handle_connection(listen_socket)
      loop(Acceptor.(listen_socket))
    end
  end
end
```

A brief note on syntax: the dot notation `Actor.e` represents a *local* expression in Chorex. The code for such expressions is stored on and runs on the named actor, and nowhere else. The examples so far have shown that variables, function calls, and non-basic expressions (e.g., `Alice.(2+2)` in Section 1) can appear after a dot. Throughout the paper, we enclose located variables in parentheses (`Alice.(x)` rather than `Alice.x`) as a stylistic choice, for compatibility with the Elixir autoformatter `mix format`. Both styles are equal to Chorex.

Since the Acceptor helper function spawns a choreography, it is useful to see its code:

```
defmodule Tcp.AcceptorImpl do
  use Tcp.ListenerChor.Chorex, :acceptor

  @impl true
  def accept_and_handle_connection(listen_socket) do
    {:ok, socket} = :gen_tcp.accept(listen_socket)
    Chorex.start(
      Tcp.HandlerChor.Chorex,
      %{Handler => Tcp.HandlerImpl, TcpClient => Tcp.ClientImpl},
      [socket]
    )
  end
end
```

This module first imports the choreography above and specifies the actor role that it plans to implement (`:acceptor`). Within the helper function, our Acceptor waits for a connection request and then calls `Chorex.start` to invoke a choreography named `Tcp.HandlerChor`. If anything goes wrong along the way, this Acceptor will exit cleanly and bring the Listener down as well. The other arguments to `Chorex.start` are a map from actor roles to module names and a list of arguments to the HandlerChor's `run` function. In this case, the only argument is the socket.

The second choreography describes an interactive loop. First, the AppHandler initializes a dictionary to track the total bytes sent by the client. Inside the loop, the Client sends a message to the AppHandler and the AppHandler updates its state, decides whether to continue, and sends an appropriate reply to the Client:

```
defmodule Tcp.HandlerChor do
  import Chorex

  defchor [AppHandler, TcpClient] do
    def run(TcpClient.(sock)) do
      loop(AppHandler.(%{byte_count: 0}), TcpClient.(sock))
    end

    def loop(AppHandler.(state), TcpClient.(sock)) do
      TcpClient.read(sock) ~> AppHandler.(msg)
      with AppHandler.({resp, st2}) <- AppHandler.run(msg, state) do
        if AppHandler.continue?(resp, st2) do
          AppHandler.fmt_reply(resp) ~> TcpClient.(resp)
          TcpClient.send_over_socket(sock, resp)
          loop(AppHandler.(st2), TcpClient.(sock))
        else
          AppHandler.fmt_reply(resp) ~> TcpClient.(resp)
          TcpClient.send_over_socket(sock, resp)
          TcpClient.shutdown(sock)
          AppHandler.ack_shutdown()
        end
      end
    end
  end
end
```

This choreography uses an `if` expression: one actor (AppHandler) makes a choice that determines the future of the conversation. The other actor thus needs *knowledge of choice* to decide which branch to take in its own code. By default, Chorex sends a message to every

other actor in a choreography whenever an `if` branch is taken. To restrict to a subset of actors, programmers can insert a `notify:` annotation. Chorex statically checks whether notify annotations include enough participants; details are in Section 3.1.

2.2 Secure Remote Password

Our second example is a choreography for the Secure Remote Password (SRP) protocol [55], a zero-knowledge-proof [30] authentication method. This choreography drives a simple command-line application that lets one user register and then serves login requests:

```
iex(1)> ZkpLogin.register_srp()
[New User SRP] username: alice
[New User SRP] password: bob
Server responds {:registered, "alice"}
Client responds :registered
:ok

iex(2)> ZkpLogin.login_srp()
[Login] username: alice
[Login] password: notbob
Server responds {:fail, :reject_client_digest}
Client responds {:fail, :server_rejected_digest}
```

Inside the choreography, there are two `run` functions corresponding to the registration and login phases. Chorex distinguishes these functions by their arity: the registration function expects a username, password, and `:register` token, and the login function expects no input. The code below shows the registration `run` function:

```
defmodule Zkp.SrpChor do
  import Chorex

  defchor [SrpServer, SrpClient] do
    def run(SrpClient.{uname, pwd}, SrpServer.{:register}) do
      SrpServer.get_params() -> SrpClient.{salt, g, n}
      with SrpClient.(v) <- SrpClient.gen_token(uname, pwd, salt, g, n) do
        SrpClient.{uname, salt, v} -> SrpServer.{uname, salt, v}
        if SrpServer.register(uname, salt, v) do
          SrpServer.{:registered, uname}
          SrpClient.{:registered}
        else
          SrpServer.{:error, :no_registration, uname}
          SrpClient.{:error, :no_registration}
        end end end
    end end

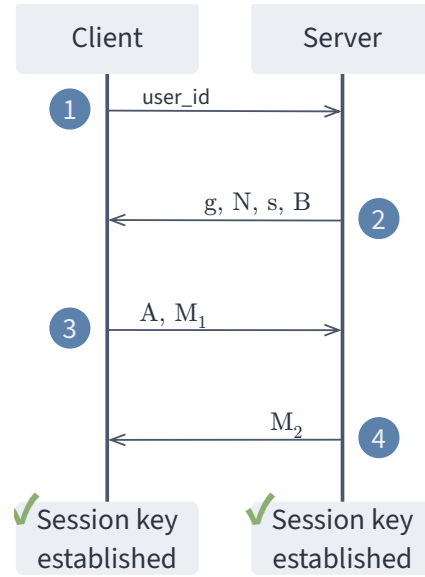
    def run() do ... end # login
  end end
```

An important property of SRP is that the password never leaves the client machine. This property is straightforward to verify by inspecting the choreography and confirming that the located value `SrpClient.pwd` never reaches another actor. In a traditional, distributed SRP implementation, such as `srp-elixir` [46], the relevant code is harder to audit because communication logic is split across files.

During the login phase of SRP, there are several rounds of communication that take place between server and client to establish a session key K . The choreographic function that implements these exchanges is roughly twice as long as the registration function yet uses

1. Client sends `user_id` to server.
2. Server finds user's salt s and auth key v , uses constants g, N to compute $k = \text{hash}(g, N)$, generates random secret b , computes $B = kv + g^b$, and sends g, N, s, B to client.
3. Client generates random secret a and computes several values, including a session key:

$$\begin{aligned} A &= g^a \\ k &= \text{hash}(g, N) \\ u &= \text{hash}(A, B) \\ K &= (B - kg^x)^{a+ux} \\ M_1 &= \text{hash}(A, B, K) \end{aligned}$$
 Client sends A, M_1 to server.
4. Server computes $K = (Av^u)^b$ and checks that $\text{hash}(A, B, K) = M_1$. Server sends $M_2 = \text{hash}(A, M_1, K)$ to confirm session key.



■ **Figure 3** Secure Remote Password login protocol to find key K . Choreography in supplement.

similar language features (`with`, `~>`, `if`), so we defer it to supplementary material. It is enough to say that comparing the choreography against a high-level algorithm description (Figure 3) is more feasible and more amenable to machine verification than comparing to a traditional codebase. For example, the choreography has exactly four communication terms (`->`), matching the figure. Of course it would be best to verify the protocol directly perhaps via CPSA [20] or Tamarin [3], but choreographies are a lightweight and general-purpose step toward bridging application code and protocol logic.

2.3 Discrete Logarithm

As a third example, we have implemented a zero-knowledge proof protocol to convince a verifier that the prover knows the logarithm of a number in a finite field. The choreography is approximately 70 lines long and is included in supplementary material.

A noteworthy aspect of this choreography is that it uses two closely-knit helper functions. An outer function loops through several rounds of challenge and verification. An inner function handles the logic of a single round. Compiling these functions to a core language that supports restarts was an encouraging stress-test for Chorex.

2.4 More Examples: Higher-Order, Out-of-Order, Persistent State

Chorex has several other features inspired by prior work on choreographies. For one, it supports first-class choreographic functions. The program in Figure 4, adapted from the Pirouette paper [32], has a `run` function that executes either a one-buyer or two-buyer bookseller scenario (both inspired by work on session types [6, 33]) by passing one function to another function.

The two function references, `@two_party/1` and `@one_party/1`, are prefixed with an `@` sign so that Chorex can add implicit arguments during compilation. See Section 3.1 for details. The suffix `/1` describes function arity. Elixir would require this suffix and an `&`-sign prefix, so Chorex syntax should not be too foreign to its intended audience.

```

defchor [Buyer, Contributor, Seller] do
  def run(Buyer.(include_contributions?)) do
    if Buyer.(include_contributions?) do
      bookseller(@two_party/1)
    else
      bookseller(@one_party/1)
    end
  end
end

def bookseller(f) do
  Buyer.get_book_title() ~> Seller.(the_book)
  with Buyer.(decision) <- f.(Seller.get_price("book:" <> the_book)) do
    if Buyer.(decision) do
      Buyer.get_address() ~> Seller.(the_address)
      Seller.get_delivery_date(the_book, the_address) ~> Buyer.(d_date)
      Buyer.(d_date)
    else
      Buyer.(nil)
    end
  end
end

def one_party(Seller.(the_price)) do
  Seller.(the_price) ~> Buyer.(p)
  Buyer.(p < get_budget())
end

def two_party(Seller.(the_price)) do
  Seller.(the_price) ~> Buyer.(p)
  Seller.(the_price) ~> Contributor.(p)
  Contributor.compute_contrib(p) ~> Buyer.(contrib)
  Buyer.(p - contrib < get_budget())
end
end

```

■ **Figure 4** Higher-order choreographic function for two classic bookseller scenarios.

269 Another important feature is support for out-of-order *message receives* to avoid commu-
 270 nication bottlenecks. In the following example, the two messages sent to `MainServer` arrive
 271 when they are ready—not necessarily in the order they appear in source code:

```

272 defmodule OooChor do
273   defchor [KeyServer, MainServer, ContentServer, Client] do
274     def run() do
275       ContentServer.getText() ~> MainServer.(txt) # may arrive 2nd
276       KeyServer.getKey() ~> MainServer.(key)      # may arrive 1st
277       ...
278     end
279   end
280 end
281 end
282

```

283 This feature is inspired by the Ozone paper [42]. We must acknowledge, however, that
 284 the Ozone calculus (O_3) is far more expressive than Chorex as it allows reordering of any
 285 expressions that do not have a data dependency. Chorex will not reorder two sends from the
 286 same actor, nor will it move expressions in or out of if branches.

287 Chorex also supports persistent state across instances of a choreography. For example,
 288 one Seller can manage a bookstore that several buyers interact with concurrently in several
 289 distinct choreographies. Similar to prior work on objects with multiple owners [14, 29], the
 290 Chorex approach is to decouple the state from the choreography itself.

2.5 Future Work

Chorex does not yet support census polymorphism [4], agreement types [37], or multiply-located values [4], but these features are on the roadmap. See Section 6 for further discussion of the related work landscape.

3 Elements of Chorex

Chorex is designed to advance research on choreographic programming by integrating with the rich Elixir/Erlang ecosystem for distributed systems. Key principles of the language design include the following:

- Maintain a smooth Elixir workflow by implementing the choreography DSL and projection to actor interfaces (known as endpoint projection, EPP, in the choreography literature) through metaprogramming.
- Manage actors using an Elixir supervision tree.
- Provide custom mailboxes and control stacks for actors to handle out-of-order messages, messages from the supervisor, and recovery.

Furthermore, Chorex draws inspiration from Elixir syntax and programming idioms, which has shaped the “look and feel” of the language as seen in Section 2.

This section explains user-facing aspects of the language design, including how to write a choreography (Section 3.1), how to implement an actor (Section 3.2), and the framework for monitoring and supervision (Section 3.3).

3.1 The Choreography Language

To write a Chorex choreography, first define a module, import the DSL, and use `defchor`:

```
defmodule SampleChor do
  import Chorex # introduces the Chorex language

  defchor [Alice, Bob, Carol] do
    ...
  end
end
```

The `defchor` macro expects two input forms: a list of actor names (all capitalized, to match the Elixir convention for module names) and a block of code. It outputs a standard Elixir module named `Chorex` that has code for actors. Thus, expanded code has the following shape:

```
defmodule SampleChor do
  import Chorex

  defmodule Chorex do
    ...
  end
end
```

Other components must refer to this macro-defined module `SampleChor.Chorex`. For instance, actor implementations must import it. (`Tcp.AcceptorImpl` in Section 2.1 is one example.)

A `defchor` form contains a sequence of function definitions. One function named `run` must be included; this function is the entry point to the choreography. No other definition forms are allowed. Ultimately, a choreography is a static artifact; it has no direct manifestation at runtime. Hence there is no way for a choreography to store variables or state of its own. In order to have meaning, code from a choreography must make its way to an actor.

XX:10 Chorex: Restartable, Language-Integrated Choreographies

Variables in a choreography must be *located* at an actor, using the syntax `Actor.(var)`. (Wrapping the variable name in parentheses is optional but preferred in Chorex.) This rule includes arguments to standard functions, as in the following snippet:

```
def some_function(Alice.(arg), Bob.(arg))
```

Before it can run, this function requires that the actor `Alice` has a field named `arg` and the actor `Bob` has a field named `arg`. Although the fields happen to have the same name (`arg`), they refer to two distinct (but possibly equivalent) values located on two different actors.

There is one exception to the rule that every variable must be located. Arguments to a higher-order function, such as `bookseller` from Section 2.4, are not located.

Elixir encourages the use of overloaded functions that each perform a specific job rather than functions that do one of several jobs after inspecting their input (via `match`, `if`, etc.). Chorex supports overloaded functions as well. The two `run` functions in the SRP choreography (Section 2.2) are one example. Cases in an overloaded function must have distinct signatures through either the number of arguments or the pattern-matching shape of those arguments. Crucially, the case-distinction must hold after a choreographic function is projected to actor code. This is why the SRP registration function expects a token `:register` located on the Server:

```
defchor [SrpClient, SrpServer] do
  # register
  def run(SrpClient.({username, password}), SrpServer.(:register)) do
    ...
  end

  # login
  def run() do ... end
end
```

Without this extra argument, the Client projection would be fine but the Server projection would contain two `run` functions that both expect zero arguments. Such ambiguity currently results in a runtime error.

373 Sending and Receiving Messages

Arrow notation (`send ~> recv`) sends a value from one actor to another. In Chorex, the sender can prepare any located expression. Variables, function calls, arithmetic, and so on are all valid on the left side of a send. The receiver can use Elixir pattern matching [23] to deconstruct the message. For example, the following is a valid send expression:

```
Alice.(:answer, 42) ~> Bob.(:answer, the_answer)
```

381 Conditionals and Knowledge of Choice

Chorex repurposes `if` expressions from Elixir for choreographic conditionals (with one change: Chorex requires an `else` branch). In the following example, `Alice` is the *deciding actor* for this conditional, as the block hinges on the result computed at `Alice`. `Bob` expects a *knowledge of choice* message to know what branch to travel down in projected code:

```
if Alice.make_decision() do
  Alice.yes() ~> Bob.(d1)
  Bob.report(d1)
else
  Alice.no() ~> Bob.(d2)
  Bob.report(d2)
end
```

By default, Chorex sends messages to *every other actor* in the choreography to share knowledge of choice. The messages contain a Boolean value (`true` or `false`). To limit the notified actors, a programmer merely needs to add a `notify:` annotation:

```

398
399 if Alice.make_decision(), notify: [Bob, Carol] do
400   ...
401 else
402   ...
403 end

```

When a `notify:` annotation is present but fails to include all actors that participate in the two `if` branches, Chorex raises a compile-time error. In the example below, `Carol` needs knowledge of choice (so that projected code can decide what to call: `foiled()` or `success()`) but `Carol` is not notified:

```

409
410 4 defchor [Alice, Bob, Carol] do
411 5   def run(Alice.(msg)) do
412 6     if Alice.decrypt(msg, priv_key()), notify: [Bob] do
413 7       Bob.notify_success()
414 8       Carol.foiled()
415 9     else
416 10      Bob.notify_failure()
417 11      Carol.success()
418 12    end
419 13  end
420 14 end
421

```

Error output:

```

423
424 == Compilation error in file bad_branch.ex ==
425 ** (CompileError) bad_branch.ex:6:
426   Branches differ for actor Elixir.Carol; 'if' block needs to notify
427

```

Unlike prior work (e.g. [32]), an `if` in Chorex need not appear in tail position. Other expressions can follow an `if` expression.

Binding Variables

An actor can bind variables using Elixir's `with` notation. In the following example, `Alice` computes a value and stores the result in the located variable `x`:

```

433
434 with Alice.(x) <- compute_value() do
435   ...
436 end
437

```

Function Returns

Each function defined in a choreography gets projected to several versions, one for each actor in the choreography. Consequently, every actor is able to call every function. To simplify questions about what a function should return and to whom it should return, every function call computes a value at every actor.

First-Class Choreographic Functions

Chorex allows functions that take other functions as input, such as `work` below. To send a function as a value, the function name must be prefixed with an `@` sign and suffixed with its arity (to disambiguate overloads):

```

447
448 def run() do
449   if Alice.need_stones() do
450     work(@gather_stones/1)
451   else
452     work(@cast_away_stones/1)
453   end
454 end
455
456 def work(f) do
457   ...
458 end
459
460 def gather_stones(Bob.(x)) do
461   ...
462 end
463
464 def cast_away_stones(Bob.(x)) do
465   ...
466 end
467

```

Chorex implicitly adds one extra argument, representing choreography state, to occurrences of first-class functions. Thus `@gather_stones/1` compiles to `&gather_stones/2`.

Error Rescue and Restarts

Chorex adapts Elixir's `try/rescue` blocks to handle errors that may arise in actor code. (Unlike in Elixir, `rescue` declares a block and not a match clause.) If Alice or Bob were to fail in the following `try` block, both actors would execute the `rescue` block with the failing actor restored via a new actor instance with recovered state:

```

477
478 try do
479   Alice.dangerous_operation() ~> Bob.(x)
480   Bob.success(x)
481 rescue
482   Alice.safe_operation() ~> Bob.(x)
483   Bob.fallback(x)
484 end
485

```

Like Chorex `if` expressions, `try` expressions do not need to appear in tail position. The `Recover` choreography in Section 1 presented a non-tail rescue block.

Figure 5 presents a flowchart description of `try/rescue` semantics. At the start of the `try` block, every actor checkpoints its state. This checkpoint is used to enter the `rescue` block, if needed. During execution of the `try` block, a runtime monitor watches for crashed processes. If all goes well for a single actor, it pauses at the `end` of the `try/rescue` in case some actor crashes. If a failure occurs, the monitor restarts the failed actor, sends topology updates to other actors, and unwinds actors to the checkpointed state.

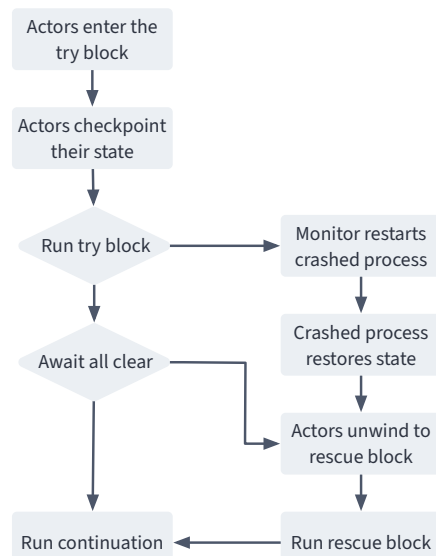


Figure 5 Chorex `try/rescue` logic.

3.2 Actor Interface

By itself, a choreography cannot run. A choreography merely coordinates the interactions between actors. It does not (and should not) define all the computations that take place within each actor process.

In order to start a Chorex choreography, driver code must supply implementation modules for each of the actors. Section 2.1 presented an example driver; module `Tcp.AcceptorImpl` called `Chorex.start` with a mapping from actor names to module names. The `AcceptorImpl` module also happens to be an implementation module, as it implements to role named `Acceptor` from the top-level TCP choreography. We repeat the first two lines of the `Acceptor` implementation below:

```
defmodule Tcp.AcceptorImpl do
  use Tcp.ListenerChor.Chorex, :acceptor
  ...
end
```

During choreography projection, Chorex outputs one module for each actor listed in the `defchor` form. These actor modules contain one version of all the communication code from the choreography. These modules also specify an Elixir interface, technically called a *behaviour*, that the actor implementation must satisfy. A behaviour is a list of functions. In Chorex, the functions are derived implicitly by the choreography.

For example, the choreography code below makes several assumptions about the actors `Alice` and `Bob`. The first line calls a function `get_money()` located on the `Alice` actor, the second line asks `Bob` to `fetch_apples`, and the third line asks `Alice` to `fetch_sugar` and `bake_pie`:

```
Alice.get_money() ~> Bob.(payment)
Bob.fetch_apples(payment) ~> Alice.(apples)
Alice.bake_pie(apples, fetch_sugar())
```

An implementation for `Alice` must implement each function in the wishlist, similar to the outline below. `Alice` is free to implement other helper functions as well:

```
defmodule AliceImpl do
  use DemoChor.Chorex, :alice

  def get_money(), do: ...
  def fetch_sugar(), do: ...
  def bake_pie(apples, sugar), do: ...
end
```

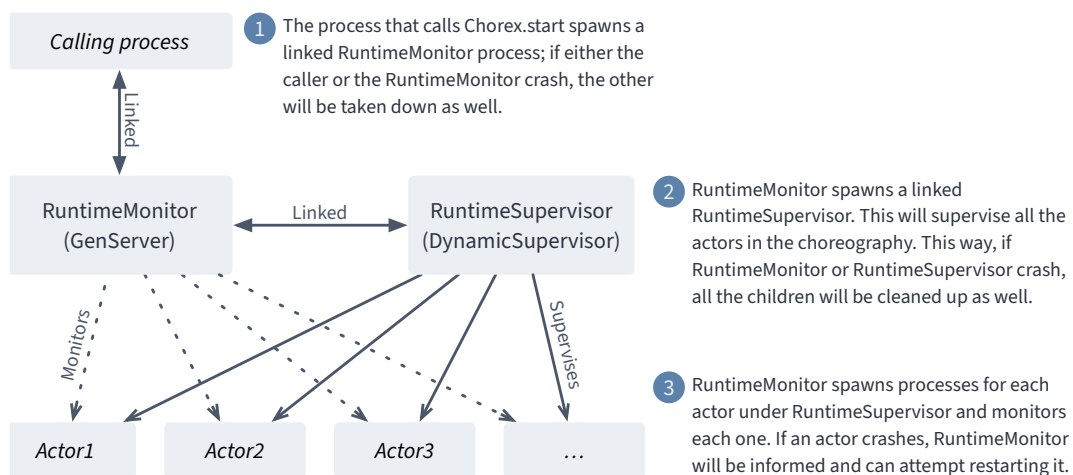
An implementation for `Bob` would `use` the role `:bob` and provide a function `fetch_apples` that accepts one argument.

Chorex provides guidance to implementation modules in the form of langserver tooltips, illustrated in Figure 1a. These are enabled through metaprogramming and cooperation with Elixir's behaviour mechanism. In particular, the line `use DemoChor.Chorex, :alice` expands at compile time to code that glues this implementation module to the choreography's projected module for `Alice`.

3.3 Supervision Protocol

Recovery from actor crashes is a key contributions of Chorex. To the best of our knowledge, Chorex is the first choreographic language to support exception handling (`try/rescue`).

XX:14 Chorex: Restartable, Language-Integrated Choreographies



■ **Figure 6** Chorex choreographies initialize one monitor and one supervisor to manage actors.

549 Monitoring and Supervision Setup

550 Chorex leverages Elixir/Erlang process monitoring facilities to supervise choreographic actors.
551 In Elixir, when a process **A** monitors another process **B** and **B** exits, process **A** receives a
552 message that describes how **B** terminated (e.g., normal exit vs. crash). Chorex creates
553 monitoring links to build a supervision tree in the style of Figure 6 every time a program
554 starts a choreography.

555 The supervision tree adds two processes to a choreography, in addition to the calling
556 process and actor processes: a `RuntimeMonitor` and a `RuntimeSupervisor`. The Monitor, Su-
557 pervisor, and calling process are linked together such that if any one process crashes, the
558 entire choreography exits cleanly. The Supervisor's job is to perform this cleanup on the
559 actors. The Monitor's job is to watch for actors that crash, restart them, and send updates
560 to reconfigure the network topology.

561 Technically, the Supervisor implements an Elixir behaviour called `DynamicSupervisor` [11].
562 The Monitor implements the so-called `GenServer` behaviour [12]. Each actor is a `GenServer`
563 as well; Section 4 explains what a `GenServer` does and why the choice matters for actors.

564 Restarting a Crashed Process

565 When an actor process enters a `try/rescue` block, it creates a checkpoint that includes the
566 control stack, message inbox, and bindings for local variables. The actor sends the control
567 stack and variable bindings to the Monitor process. If a crash occurs, the Monitor spawns a
568 new process, which has no state initially, and installs missing pieces.

569 Storing checkpoints in the Monitor is a convenient choice. The Monitor could easily move
570 this data to the filesystem, a database, or even another process. The key benefit of the
571 Monitor is that it is linked to the actors and to the calling process; if the Monitor goes down,
572 the choreography exits and there is no need to recover actors anymore.

573 After reinstating an actor process, the Monitor alerts all other actors that a crash has
574 occurred. Actors handle this message by unwinding their control stacks to the frame that
575 transfers control to the `rescue` block.

Can a Restarted Process Miss Messages?

For an actor that eventually crashes, there is a potential danger of missing messages. Any messages that arrive while the actor is executing the `try` block go to its inbox, but not to its checkpointed state. (Updating the checkpoint for every receive would entail a great deal of communication with the Monitor.) However, this concern is not an actual threat. Any messages that arrive window must have originated from other actors within the same `try` block because of how Chorex synchronizes actors. All processes enter the `try` simultaneously, and all must complete the `try` before they can continue executing. This design protects against missing messages. It also ensures that actors do not terminate prematurely and fail to unwind if the need for a `rescue` arises.

Handling Out-of-Band Recovery Messages

Actor processes could inherit a call stack and message inbox from Elixir (technically, from the GenServer behaviour). Chorex manages its own call stacks and inboxes to enable recovery, particularly for stack unwinding and out-of-band message receives.

For unwinding, the Monitor broadcasts that the most recent `try` block failed along with a token indicating which stack frame actors must unwind to. Every Chorex actor process has a message handler dedicated to receiving such messages. These handlers interrupt normal execution to effect a reset.

4 Implementation

Under the hood, Chorex translates choreographies to sets of sets of message-passing functions. Each function in the choreography spins off into several variants, one for each actor. Each variant is in turn split into several functions (distinguished by generated tokens) corresponding to receive-separated chunks of the choreography. Putting it all together required a number of significant components: a tailored message format (Section 4.1), a method of organizing component functions (Section 4.2), and an implementation of the compiler as a metaprogram to maximize host-language integration (Section 4.3). We conclude this section with lessons learned for related implementation efforts (Section 4.3.2).

4.1 Chorex Message Format

There are two categories of messages that a Chorex actor needs to handle: choreography messages and control messages. *Choreography messages* go between actors, and originate from the sends and receives in a choreography. *Control messages* come from the Chorex runtime; these messages propagate knowledge of choice, notify actors of crashes and network updates, and synchronize actor execution of `try/rescue` blocks.

Most messages have a standard 3-tuple format:

```
MSG = {message_type, civ_token, payload}
```

The `message_type` field may have one of the following values:

- `:chorex`, for a choreography message. The corresponding payload is a message (Elixir value) created by one actor and intended for another actor.
- `:choice`, for knowledge-of-choice. Payload is a Boolean choice value.
- `:revive`, for error recovery. This type of message is a 2-tuple; it does not include a CIV token. A process that was created to handle a crash receives this message. Payload is the new actor state to install.

XX:16 Chorex: Restartable, Language-Integrated Choreographies

620 ■ `:recover`, for error recovery. This type of message is sent to actors that did not crash, and
621 asks them to unwind to the nearest `rescue` point. Payload represents the new network
622 configuration.
623 ■ `:barrier`, for synchronization at `try` blocks. Indicates that all actors successfully completed
624 the `try` block and may proceed into the continuation.

625 The `civ_token`, inspired by Ozone [42], preserves communication integrity in the presence
626 of out-of-order messages. Each CIV token is a 4-tuple:

```
627 CIV = {session_token, metadata, sender, receiver}
```

630 The `session_token` is a UUID generated when the choreography is instantiated (during
631 `Chorex.start`). Every instance of a choreography has its own session token. The `metadata` field
632 describes the source-code location within the choreography where this message originated
633 (i.e., the original `send` $\rightsquigarrow$ `recv`). Crucially, both parties involved in the message receive equal
634 `metadata` values, and different `send` $\rightsquigarrow$ `recv` sites lead to different `metadata` values. The `sender`
635 and `receiver` components are the names of the actors involved. All together, a CIV token
636 ensures that a message goes only to the intended destination.

637 4.2 Actors as Server Processes

638 Chorex projects actors to Elixir `GenServer` behaviors [12, 19] rather than straight-line
639 processes. This choice solves the following problems with default Elixir processes:

- 640 1. no ability to receive messages out-of-order,
- 641 2. no way to prioritize messages from the Monitor process (Figure 6),
- 642 3. no way to supervise for crashes.

643 The first two issues arise because there is no way to scan an inbox for all messages that
644 match a given pattern. Only the first message can be selected. The third issue is an artifact
645 of how supervision trees work in Elixir.

646 `GenServers`, by contrast, can define message handlers that fire as soon as a matching
647 message arrives, and they cooperate in supervision trees. `GenServers` have their own
648 drawbacks as well, e.g., complicated variable scope, but Chorex works around these problems.

649 GenServer Primer

650 `GenServer` (short for “Generic Server”) is an Erlang library that makes it easy to build
651 processes that manage state and can respond to ad-hoc messages. Elixir inherits `GenServers`
652 from Erlang.

653 To behave as a `GenServer`, a module must define an `init` function that returns a value
654 representing the server state, and callbacks to handle incoming messages. There are three
655 kinds of callbacks that deal with messages: `handle_call`, `handle_cast`, and `handle_info`. These
656 callbacks can expect at least a message and state value as input, and must return a new
657 state value. A `handle_call` receives a process ID as well, and must include a reply to this
658 process in its return value.

659 As an example, the `GenServer` in Figure 7 implements a counter. The initial state is the
660 number `start_count`. One callback, `handle_cast`, awaits messages with the tag `:increment` and
661 a numeric value; it adds the input value to the counter state. Another callback, `handle_call`,
662 awaits `:get_count` messages. It logs information about the request and returns a 3-tuple with
663 two copies of the state. One copy is a reply to the `sender` process and the other copy is
664 the new state (same as before the call). To start an instance of this module with an initial


```

defmodule Counter do
  use GenServer

  @moduledoc "hello"

  # set up initial count value
  def init(start_count) do
    {:ok, start_count}
  end

  def handle_cast(:increment, count) do
    {:noreply, count + 1} # last element is new state
  end

  def handle_call(:get_count, sender, count) do
    IO.puts("Got a request for the count from sender #{inspect sender}")
    {:reply, count, count} # middle element gets sent to sender
  end
end

```

■ **Figure 7** Example GenServer that implements a counter.

665 count of 0, one calls `GenServer.start(Counter, 0)`. This invokes the `init` callback with 0 as
 666 the `start_count` argument.

667 Multiple processes can interact with a GenServer at the same time. The GenServer
 668 handles each incoming message, one at a time. This behavior enforces linearity and ensures
 669 that concurrent processes can interact with shared state in a safe, coherent way.

670 Challenges

671 The key challenge of GenServers as a target for choreography projection is that every possible
 672 message that an actor can receive must be anticipated by a callback. An actor cannot
 673 be implemented with a single function; it must be split across several functions as shown
 674 in Figure 8. Consequently, variables defined earlier in a choreography must become part of
 675 the GenServer state in order to reach later parts of the choreography.

676 A second challenge is that actors can no longer use the Elixir call stack to handle function
 677 calls. Chorex must manage its own actor-specific control stacks.

678 Correct Scope via Live Variable Analysis

679 Consider a simple choreography in which `Alice` and `Carol` both send messages to `Bob`:

```

680 Alice.one() ~> Bob.(x)
681 Carol.two() ~> Bob.(y)
682 Bob.(x + y)
683
684

```

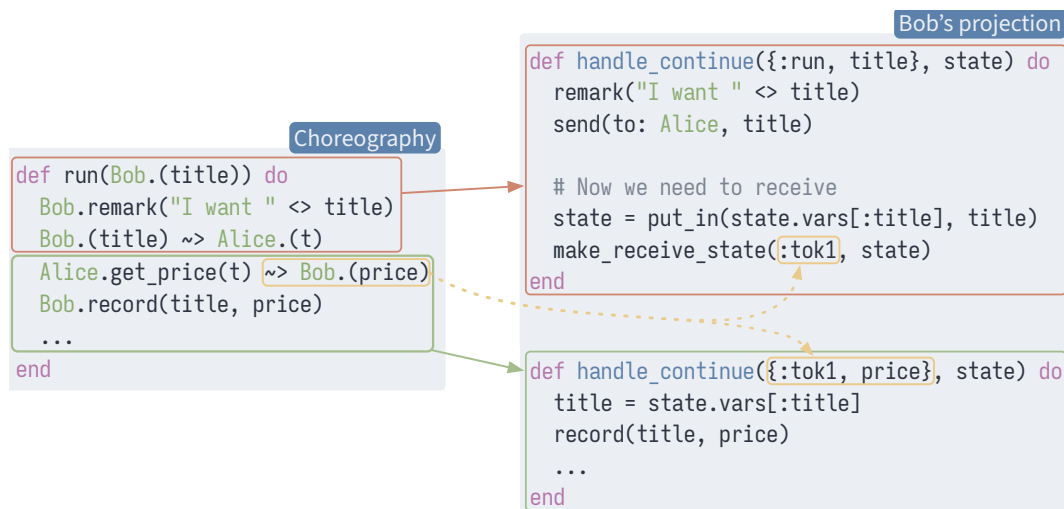
685 Projection for the `Bob` actor introduces two callbacks, one for each receive. A direct but
 686 incorrect projection would use the variables `x` and `y` directly. This is wrong because `x` is not
 687 in scope for the second callback, which needs to return a sum:

```

688 # Bob projection, version 0
689 def handle_info({Alice, x}, state), do: ...
690 def handle_info({Carol, y}, state) do
691   x + y # WRONG
692 end
693
694

```

XX:18 Chorex: Restartable, Language-Integrated Choreographies



■ **Figure 8** Projection splits actor code into a set of callbacks: one per receive.

Chorex builds a better projection by tracking the set of live variables through a choreography. At runtime, Chorex stores a map from variable to values in GenServer state. The following code outlines the Chorex projection for Bob:

```
# Bob projection, version 1
def handle_info({Alice, x}, state) do
  state = put_in(state.vars[:x], x)
  ...
end

def handle_info({Carol, y}, state) do
  x = state.vars[:x]
  x + y
end
```

Determining free and live variable analysis in Elixir has some subtleties because of how match expressions work in Elixir. For example, the expression `[x, ^y, x] = make_list()` introduces `x` as a new variable, (even though `x` appears twice in the pattern) but `y` is *pinned* with the `^` prefix, meaning that `y` is free in this expression. Such expressions are ubiquitous in Elixir. Chorex does reuse the Elixir tree-walking API to traverse code, but it implements a custom set of rules to distinguish free and bound variables.

Managing Receives and Function Calls

The second challenge of projection to GenServers is that function calls in a choreography no longer map cleanly to function calls in the Elixir output. An actor might call a function that receives several messages; each receive will introduce a new callback. In the example below, the function `test_system` receives one message for each of the actors `Mike` and `Joe`:

```
def run() do
  Joe.(begin) ~> Mike.(start_message)
  with Joe.(response) <- test_system() do
    Joe.(String.length(response))
  end
end

# ECOOP'25 submission: unfortunate page break
```

```

731 def test_system() do
732   Joe.("Hello Mike") ~> Mike.("Hello " <> my_name)
733   Mike.("Hello Joe, you said #{my_name}") ~> Joe.(reply)
734   Joe.("Received " <> reply)
735 end

```

737 To recover typical call/return behavior, each actor tracks a stack of call frames in the
 738 GenServer state. When a callback finishes, the GenServer inspects the top stack frame to
 739 decide where to go next. It then invokes a next callback, and this jump corresponds to a
 740 return in the source language.

741 When projecting the `run` function above, Chorex does not know whether the call to
 742 `test_system` will perform any receives. (Higher-order functions make it impractical to statically
 743 track which functions do receive.) At each function call, Chorex thus creates a unique token
 744 to identify the call site and its continuation. This token is used in two places: first, it goes
 745 onto the control stack in the GenServer state; second, it appears in the argument specification
 746 of the callback that holds the continuation code.

747 Summary: Actor State

748 To sum up, each Chorex actor keeps the following state:

- 749 ■ a queue of choreography messages to be processed,
- 750 ■ a stack of control frames to recover receives and function calls,
- 751 ■ a map of live variables,
- 752 ■ the `session_token`,
- 753 ■ a map representing the network topology, and
- 754 ■ a reference to the implementing module.

755 This is all the information that an actor needs to do its job. This state gets passed
 756 between every message handler that the GenServer has, so at all times these items are
 757 available to the runtime.

758 In addition to the common state layout, Chorex actors share some functionality via a
 759 runtime module:

- 760 ■ push new messages on a queue as they arrive,
- 761 ■ inspect the control stack to determine which message in the inbox is needed next, and
- 762 ■ respond to crash notices and unwind their stacks accordingly.

763 4.3 Projection via macro expansion

764 Chorex uses Elixir's macro system to embed a choreography language. Macros expand during
 765 compilation, which allows Chorex to perform static checks such as sufficient knowledge-of-
 766 choice propagation. Macros are also part of the standard Elixir toolchain, which makes for a
 767 seamless workflow. No extra build steps are needed.

768 Within its macro implementation, Chorex reuses many affordances of the host language.
 769 Actors compile to GenServer processes. Local expressions get lowered as-is, which means that
 770 choreographies may become more expressive as Elixir improves over time. Macro hygiene
 771 provides a stable base for this reuse. Chorex users do not have to worry about macro
 772 implementation leaking out, and Chorex itself is (in principle) macro extensible.

773

4.3.1 Defchor Internals

774 Consider a simple choreography:

```

775
776 defmodule MyChor do
777   defchor [Alice, Bob] do
778     def run() do
779       Alice.one() ~> Bob.(x)
780       Alice.two() ~> Bob.(y)
781       Bob.(x + y)
782     end
783   end
784 end
785

```

786 `defchor` is a macro that takes a list of actor names and a block of choreography code.
 787 `defchor` runs a projection function over the choreography body for each of the actors. The
 788 projection function takes an actor name and a sequence of expressions and returns three
 789 values:

- 790 ■ a sequence of expressions, representing the actor's view of the expressions;
- 791 ■ a list of function clauses (`defchor` will splice these into the `GenServer` for the actor); and
- 792 ■ a list of function specifications, required of actor implementations.

793 With these pieces, the `defchor` macro can generate the code for a module for each
 794 actor. For the example above, `defchor` creates a module for `Alice` that has a single function
 795 called `run` that sends two messages to `Bob`. Additionally, `defchor` outputs glue code so that
 796 an implementation for `Alice` can begin with `use MyChor.Chorex, :alice` to receive a list of
 797 requirements; the specific requirements are functions named `one` and `two` (with proper arity).
 798 The `Bob` module gets two functions for handling each of the two messages it receives. Actor
 799 implementations for `Bob` do not need to create any functions; they have no obligations.

800 Elixir AST nodes includes metadata about source code, including line and column numbers.
 801 Chorex uses this metadata whenever possible in expanded code to ensure that error messages
 802 get reported in terms of the source language. For example, for the following faulty code:

```

803
804 Alice.one(bad_variable_name) ~> Bob.(x)
805

```

806 macro output causes the Elixir compiler to report a readable error:

```

807
808 error: undefined variable "bad_variable_name"
809   Alice.one(bad_variable_name) ~> Bob.(x)
810           ^

```

812 Static errors appear in IDEs that use the Elixir language server, so developers using Chorex
 813 benefit from a tight feedback loop.

814

4.3.2 Lessons and Reflections

815 At this early stage, the Chorex project appears to be a promising step for choreographic
 816 programming. The compilation from choreography to supervision trees and server processes
 817 enables a feature-rich choreography language with novel support for error handling and
 818 restarts. Its implementation as a macro facilitates actionable error messages, language-server
 819 integration, and a standard workflow. We conjecture that macro-extensibility for Chorex
 820 itself is a feasible next step.

821 `GenServers` as the compilation target enabled flexible, out-of-order and out-of-band
 822 message receives. This critical ability was well worth the pain of having to implement custom
 823 program analyses.

■ **Table 1** Overhead in TCP-inspired State Machine and ZKP-inspired Mini Blockchain

State Machine		Mini BlockChain	
try	1.01 x	try	9.65 x
try-rescue	1.01 x	try-rescue	??

One limitation in Elixir’s macro system is limited support for pattern-matching on syntax. The base language has excellent pattern matching for values. A tool similar to Racket’s `syntax-parse` [17] would make macros far easier to write and maintain.

Chorex projects choreographies into one module and requires actor implementations to provide application-specific details in a separate module. This design allows the reuse of one protocol, such as the bookseller scenario, with different implementations. While certain forms of reuse are enabled through higher-order functions and runtime arguments to `Chorex.start`, modularity offers a second level of customization. Furthermore, projecting the choreography into a behaviour module gave Chorex a natural way to reuse Elixir language tooltips.

Choral, the original choreographic language, draws a sharp distinction between projection and execution [29]. Projection is a first step in the workflow, and it outputs code that developers may wish to maintain (e.g., to interoperate with standard interfaces [38]). Having to manage two codebases, the choreography and its projection, raises important questions about *round-trip development* so that changes in the projected code can be reflected in the choreography. Chorex, and projection-at-runtime languages such as HasChor [47, 4], remove the need for round-trip development by making it impractical to edit projected code. This appears to be a step in the right direction, but more experience applying Chorex to existing code is needed to see whether the choreography language is missing features that Choral projections provide.

5 Benchmarking Restarts

The purpose of this section is to measure the overhead of `try/rescue` blocks. Entering a `try` creates a snapshot. Entering a `rescue` restores a snapshot. Both actions are potentially expensive.

We evaluate on two case-study programs (intended to represent realistic scenarios) and several microbenchmarks that stress-test aspects of scalability. and seve stress-test microbenchmarks. All experiments were conducted on a single-user Apple M1 Pro with 32 GB RAM and 10 available cores. [TODO how did we specify (or not specify, what’s the default?) core use?] For software, we used Chorex v[TODO: 0.8.10?], Elixir v1.18.0, and Erlang v27.2, all with default settings and with JIT enabled.

5.1 Case Studies

We have two case studies.

State Machine , technically it implements a text-based adventure game.

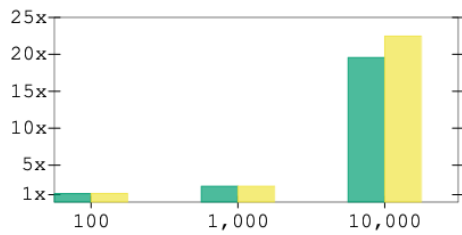
Mini Blockchain is TODO.

Table 1 presents the overhead of TODO. For State Machine, the `try` block ... For Mini Blockchain, TODO.

5.2 Microbenchmarks

TODO deep loop, Figure 9.

XX:22 Chorex: Restartable, Language-Integrated Choreographies



■ **Figure 9** Deep loop, no-split (green) and split

861 Overhead is negligible for a flat loop, without recursion in `try` blocks. At 10,000 iterations
862 the overhead of adding a `try` is 1.07 x.

863 5.3 Discussion

864 TODO

865 case studies okay

866 By far the most punishing benchmarks are those where there are nested recursive `try`
867 blocks. A structure like:

```
868 def loop(...) do
869   try do
870     ...
871     loop(...)
872   rescue
873     ...
874   end
875 end
876
877
```

878 is extremely punishing as the stack gets deeper and deeper. This can be seen in the
879 Miniblock and Deep Loops benchmarks. In contrast, a structure where the `try` is not involved
880 in the recursion, like in Flat Loops, the performance is much better:

```
881 def loop(...) do
882   do_work(...)
883   loop(...)
884 end
885
886 def do_work(...) do
887   try do
888     ...
889   rescue
890     ...
891   end
892 end
893
894
```

895 Finally, `try/rescue` seems to impose a negligible impact when there are a large number
896 (100) of actors. Since these actors are specified manually, and Chorex does not yet have
897 census polymorphism, 100 seems to be a reasonable torture test for a choreography.

Table 2 Survey of recent advances in choreographic programming. The concept of functions as values has seen wide adoption (Functional column). Other recently-proposed features have yet to permeate the landscape. Chorex adds restarts with topology updates to the feature-space.

	Functional	Restartable	Full OoO	Census Poly	Agreement- τ
Choret [5]	✓				
HasChor [47]	✓				
★ Chorex	✓	✓			
Choral [13, 29, 42]	✓		✓		
Klor [37]	✓				✓
MultiChor + ChoRus + ChoreoTS [4]	✓			✓	

6 Related Work

The design and implementation of choreographic languages has become a lively research area. Java, Haskell, Racket, Rust, and (now) Elixir all have third-party support for choreographies today, and most of these implementations appeared within the past year [4, 5, 9, 29, 37]. Choral is a notable exception with nearly a decade of engineering under its belt [28]. A first workshop on choreographies [1] and an introductory zine [2] appeared last year as well.

As Table 2 outlines, the overall expressiveness of choreographies as a protocol language is expanding. Functional (or higher-order [14, 32]) choreographies, which can use functions as first-class values, are standard. Restarts and topology changes are unique to Chorex. Fully out-of-order execution (Full OoO) lets a choreography reorder code in any way that respects data dependencies. An efficient realization of (partial) reordering is in the Ozone API [42], which compiles to Choral. Census polymorphism allows abstraction over the number of participants in a choreography, analogous to variable-arity functions [21]. MultiChor, ChoRus, and ChoreoTS (which were introduced simultaneously [4]) are the first languages to support census polymorphism. Agreement types in Klor track the participants involved in a subroutine and thus provide a compositional way to infer knowledge-of-choice annotations. Chorex provides a modicum of reordering to avoid performance pathologies; messages sent to an actor arrive as soon as they are ready, instead of queuing. We have no plans at this time to implement full reordering. The other features in Table 2 are on the agenda for future work improving Chorex.

An orthogonal dimension for choreographic languages is whether to implement choreographies as a standalone language (like Choral) or as a library. Libraries are typically easier to implement, but limited in power. For example, HasChor [47] broadcasts knowledge-of-choice to all participants—turning every conditional into a choreography-wide sync point—because it cannot perform a typical two-pass analysis (as in [32]). MultiChor and its relatives address this problem with an *enclave* mechanism that can reduce the actors in each broadcast [4]. A thesis of Chorex, and of Choret [5], is that the best implementation of all is to implement a language as a library in a meta-programmable host language such as Racket [18, 25]. Along similar lines, recent work improves HasChor with static choreography projection via a Haskell compiler plugin [36].

Theoretical foundations of choreographies have been progressing for over a decade [7, 15, 16, 32, 42, 43]. The Pirouette calculus was the blueprint we followed to start Chorex [32].

Elixir is soon to acquire a full-featured gradual type system based on set-theoretic types [8]. Release v1.18 [53] infers types from match patterns and function bodies to report certain high-confidence warnings. However, users cannot write types and the typechecker assumes the

permissive dynamic type by default, limiting the guarantees that types provide. Extending this type system to gradually check choreographies is an exciting future direction.

Lastly, we mention areas that have close ties to choreographies. Multiparty session types [33, 34] and secure multiparty computation (SMC) [44, 49] both coordinate distributed actors via a global protocol. Session types merely specify required behavior, giving developers the freedom to build a conforming implementation. One notable realization of session types is ElixirST, a typechecker for a language of Elixir processes [26]. Writing programs that conform to session types can be challenging; techniques for API generation address the implementation burden [10], similar to how Chorex choreographies generate requirements for actor modules. SMC languages avoid the conformance question by generating code from a protocol. They are effectively choreographic languages, but designed and constrained by security considerations.

Recent work on hybrid session types shows how to compose a global protocol from local, application specific protocols [27]. While Chorex allows choreographies to start other choreographies, and thus supports some composition, it cannot make static guarantees. Hybrid choreographies may be the way forward. Another closely-related work is the Corps calculus for hierarchical choreographies [31].

7 Conclusion

The essence of Chorex is a compiler from multiparty programs to stateless, message-passing processes. This compiler breaks new ground for choreographic programming with *restartable actors*, which are enabled through a checkpointing protocol and cooperative supervisor process, and with its *tight integration* to the host language, enabled by metaprogramming. It was not at all clear at the start of this project that an expressive choreographic language could be implemented as a meta-program (as opposed to a standalone compiler). The fact that Chorex works for Elixir indicates that other metaprogrammable languages, from Ruby [35] to Racket [18] to Rust [45], can follow suit by building a library for supervised processes (Section 3.3) and a functional-style server runtime (Section 4.2). Extending the Chorex compiler protocol to support multiple languages in the same toolchain, with projection to message-passing actors written in different languages, would be a worthy challenge for future work.

References

- 1 1st International Workshop on Choreographic Programming, 2024. <https://pldi24.sigplan.org/home/cp-2024>. Accessed 2025-02-28.
- 2 Ali Ali and Lindsey Kuper. Communicating chorrectly with a choreography, 2024. <https://decomposition.al/zines/#communicating-chorrectly-with-a-choreography>. Accessed 2025-02-28.
- 3 David A. Basin, Cas Cremers, Jannik Dreier, and Ralf Sasse. Tamarin: Verification of large-scale, real-world, cryptographic protocols. *SP*, 20(3):24–32, 2022. doi:10.1109/MSEC.2022.3154689.
- 4 Mako Bates, Shun Kashiwa, Syed Jafri, Gan Shen, Lindsey Kuper, and Joseph P. Near. Efficient, portable, census-polymorphic choreographic programming, 2024. URL: <https://arxiv.org/abs/2412.02107>, arXiv:2412.02107.
- 5 Alexander Bohosian and Andrew K. Hirsch. Choret: Functional choreographic programming for Racket, 2025. <https://github.com/Foundations-of-Decentralization-Group/choret>. Accessed 2025-02-26.

- 978 6 Marco Carbone, Kohei Honda, and Nobuko Yoshida. A calculus of global interaction based on
979 session types. In *DCM@ICALP*, number 3, pages 127–151. Elsevier, 2006. doi:10.1016/J.
980 ENTCS.2006.12.041.
- 981 7 Marco Carbone and Fabrizio Montesi. Deadlock-freedom-by-design: Multiparty asynchronous
982 global programming. In *POPL*, pages 263–274. ACM, 2013. doi:10.1145/2429069.2429101.
- 983 8 Giuseppe Castagna, Guillaume Duboc, and José Valim. The design principles of the Elixir
984 type system. *Programming*, 8(2), 2024. doi:10.22152/PROGRAMMING-JOURNAL.ORG/2024/8/4.
- 985 9 Kaushik Chakraborty. Unichorn, 2024. [https://share.unison-lang.org/@kaychaks/
986 unichorn/](https://share.unison-lang.org/@kaychaks/unichorn/). Accessed 2025-03-02.
- 987 10 Guillermina Cledou, Luc Edixhoven, Sung-Shik Jongmans, and José Proença. API generation
988 for multiparty session types, revisited and revised using Scala 3. In *ECOOP*, pages 27:1–27:28.
989 Schloss Dagstuhl, 2022. doi:10.4230/LIPICS.ECOOP.2022.27.
- 990 11 Elixir Contributors. Dynamic Supervisor behaviour, 2021. [https://hexdocs.pm/elixir/1.
991 13/DynamicSupervisor.html](https://hexdocs.pm/elixir/1.13/DynamicSupervisor.html). Accessed 2025-03-03.
- 992 12 Elixir Contributors. GenServer behaviour, 2021. [https://hexdocs.pm/elixir/1.12/
993 GenServer.html](https://hexdocs.pm/elixir/1.12/GenServer.html). Accessed 2024-12-16.
- 994 13 Luís Cruz-Filipe, Eva Graversen, Lovro Lugovic, Fabrizio Montesi, and Marco Peressotti.
995 Functional choreographic programming. In *ICTAC*, pages 212–237. Springer, 2022. doi:
996 10.1007/978-3-031-17715-6_15.
- 997 14 Luís Cruz-Filipe, Eva Graversen, Lovro Lugović, Fabrizio Montesi, and Marco Peressotti.
998 Modular compilation for higher-order functional choreographies. In *ECOOP*, pages 7:1–7:37.
999 Schloss Dagstuhl, 2023. doi:10.4230/LIPICS.ECOOP.2023.7.
- 1000 15 Luís Cruz-Filipe and Fabrizio Montesi. A core model for choreographic programming. *Theo-
1001 retical Computer Science*, 802:38–66, 2020. doi:10.1016/J.TCS.2019.07.005.
- 1002 16 Luís Cruz-Filipe, Fabrizio Montesi, and Marco Peressotti. A formal theory of choreo-
1003 graphic programming. *Journal of Automated Reasoning*, 67(2):21, 2023. doi:10.1007/
1004 S10817-023-09665-3.
- 1005 17 Ryan Culpepper. Fortifying macros. *Journal of Functional Programming*, 22(4-5):439–476,
1006 2012. doi:10.1017/S0956796812000275.
- 1007 18 Ryan Culpepper, Matthias Felleisen, Matthew Flatt, and Shriram Krishnamurthi. From
1008 macros to DSLs: The evolution of Racket. In *SNAPL*, volume 136, pages 5:1–5:19. Schloss
1009 Dagstuhl, 2019. doi:10.4230/LIPICS.SNAPL.2019.5.
- 1010 19 Erlang documentation authors. Documentation for the gen_server behaviour. [https://www.
1011 erlang.org/doc/apps/stdlib/gen_server.html](https://www.erlang.org/doc/apps/stdlib/gen_server.html). Accessed 2025-02-18.
- 1012 20 Shaddin F. Doghmi, Joshua D. Guttman, and F. Javier Thayer. Searching for shapes
1013 in cryptographic protocols. In *TACAS*, pages 523–537. Springer, 2007. doi:10.1007/
1014 978-3-540-71209-1_41.
- 1015 21 R. Kent Dybvig and Robert Hieb. A new approach to procedures with variable arity. *LISP
1016 and Symbolic Computation*, 3(3):229–244, 1990.
- 1017 22 Elixir Contributors. Mix, 2024. <https://hexdocs.pm/mix/1.18.2/Mix.html>. Accessed 2025-
1018 03-04.
- 1019 23 Elixir Contributors. Patterns and guards, 2024.
- 1020 24 Elixir Contributors. hex, 2025. <https://hex.pm/>. Accessed 2025-03-04.
- 1021 25 Matthias Felleisen, Robert Bruce Findler, Matthew Flatt, Shriram Krishnamurthi, Eli Barzilay,
1022 Jay A. McCarthy, and Sam Tobin-Hochstadt. The Racket manifesto. In *SNAPL*, pages 113–128.
1023 Schloss Dagstuhl, 2015. doi:10.4230/LIPICS.SNAPL.2015.113.
- 1024 26 Adrian Francalanza and Gerard Tabone. ElixirST: A session-based type system for Elixir
1025 modules. *Journal of Logical and Algebraic Methods in Programming*, 135(100891), 2023.
1026 doi:10.1016/J.JLAMP.2023.100891.
- 1027 27 Lorenzo Gheri and Nobuko Yoshida. Hybrid multiparty session types: Compositionality
1028 for protocol specification through endpoint projection. 7(OOPSLA1):112–142, 2023. doi:
1029 10.1145/3586031.

- 1030 28 Saverio Giallorenzo, Fabrizio Montesi, and Marco Peressotti. Choral: Object-oriented choreo-
1031 graphic programming, 2023. URL: <https://arxiv.org/abs/2005.09520>, arXiv:2005.09520.
- 1032 29 Saverio Giallorenzo, Fabrizio Montesi, and Marco Peressotti. Choral: Object-oriented choreo-
1033 graphic programming. *Transactions on Programming Languages and Systems*, 46(1):1:1–1:59,
1034 2024. doi:10.1145/3632398.
- 1035 30 Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive
1036 proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989. doi:10.1137/0218012.
- 1037 31 Andrew K. Hirsch. Corps: A core calculus of hierarchical choreographic programming. *CoRR*,
1038 abs/2406.01456, 2024. arXiv:2406.01456, doi:10.48550/ARXIV.2406.01456.
- 1039 32 Andrew K. Hirsch and Deepak Garg. Pirouette: Higher-order typed functional choreographies.
1040 *PACMPL*, 6(POPL):1–27, 2022. doi:10.1145/3498684.
- 1041 33 Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty asynchronous session types.
1042 In *POPL*, pages 273–284. ACM, 2008. doi:10.1145/1328438.1328472.
- 1043 34 Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty asynchronous session types.
1044 *Journal of the ACM*, 63(1):9:1–9:67, 2016. doi:10.1145/2827695.
- 1045 35 Milod Kazerounian, Sankha Narayan Guria, Niki Vazou, Jeffrey S. Foster, and David Van Horn.
1046 Type-level computations for Ruby libraries. In *PLDI*, pages 966–979. ACM, 2019. doi:
1047 10.1145/3314221.3314630.
- 1048 36 Robert Krook and Samuel Hammersberg. Welcome to the parti(tioning) (functional pearl):
1049 Using rewrite rules and specialisation to partition Haskell programs. In *Haskell*, pages 27–40.
1050 ACM, 2024. doi:10.1145/3677999.3678276.
- 1051 37 Lovro Lugović and Sung-Shik Jongmans. Klor: Choreographies in Clojure, 2024. [https:](https://github.com/lovrosdu/klor)
1052 [/github.com/lovrosdu/klor](https://github.com/lovrosdu/klor). Accessed 2024-12-03.
- 1053 38 Lovro Lugović and Fabrizio Montesi. Real-world choreographic programming: Full-duplex
1054 asynchrony and interoperability. *Programming*, 8(2), 2024. doi:programming-journal.org/
1055 2024/8/8.
- 1056 39 Fabrizio Montesi. *Choreographic Programming*. PhD thesis, IT University of Copenhagen,
1057 2013. URL: <https://www.fabriziomontesi.com/files/choreographic-programming.pdf>.
- 1058 40 Fabrizio Montesi. *Introduction to Choreographies*. Cambridge University Press, 2023. doi:
1059 10.1017/9781108981491.
- 1060 41 OMG. Sequence diagrams, 2017. <https://www.omg.org/spec/UML/2.5.1/PDF>. Accessed 2025-
1061 02-26.
- 1062 42 Dan Plyukhin, Marco Peressotti, and Fabrizio Montesi. Ozone: Fully out-of-order choreogra-
1063 phies. In *ECOOP*, pages 31:1–31:28. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024.
1064 doi:10.4230/LIPICS.ECOOP.2024.31.
- 1065 43 Mila Dalla Preda, Maurizio Gabbriellini, Saverio Giallorenzo, Ivan Lanese, and Jacopo Mauro.
1066 Dynamic choreographies: Theory and implementation. *Logical Methods in Computer Science*,
1067 13(2), 2017. doi:10.23638/LMCS-13(2:1)2017.
- 1068 44 Aseem Rastogi, Matthew A. Hammer, and Michael Hicks. Wysteria: A programming language
1069 for generic, mixed-mode multiparty computations. In *SP*, pages 655–670. IEEE Computer
1070 Society, 2014. doi:10.1109/SP.2014.48.
- 1071 45 Rust Contributors. Rust macros, 2024. [https://doc.rust-lang.org/reference/](https://doc.rust-lang.org/reference/procedural-macros.html)
1072 [procedural-macros.html](https://doc.rust-lang.org/reference/procedural-macros.html). Accessed 2024-01-17.
- 1073 46 Tiago Santos. Srp, 2019. <https://github.com/thiamsantos/srp-elixir>. Accessed 2025-03-
1074 03.
- 1075 47 Gan Shen, Shun Kashiwa, and Lindsey Kuper. HasChor: Functional choreographic pro-
1076 gramming for all (functional pearl). *PACMPL*, 7(ICFP):541–565, 2023. URL: [https:](https://doi.org/10.1145/3607849)
1077 [/doi.org/10.1145/3607849](https://doi.org/10.1145/3607849).
- 1078 48 Simon St. Laurent and J. David Eisenberg. *Introducing Elixir: Getting Started in Functional*
1079 *Programming*. O'Reilly Media, 2nd edition, 2017.
- 1080 49 Ian Sweet, David Darais, David Heath, William Harris, Ryan Estes, and Michael Hicks.
1081 Symphony: Expressive secure multiparty computation with coordination. *The Art, Science,*

1082 *and Engineering of Programming*, 7(3), 2023. doi:10.22152/PROGRAMMING-JOURNAL.ORG/
1083 2023/7/14.

1084 50 Dave Thomas. *Programming Elixir*. Pragmatic Bookshelf, 1st edition, 2018.

1085 51 Thousand Island Contributors. Thousand Island, 2025. [https://github.com/mtrudel/](https://github.com/mtrudel/thousand_island)
1086 [thousand_island](https://github.com/mtrudel/thousand_island). Accessed 2025-01-30.

1087 52 Sam Tobin-Hochstadt, Vincent St-Amour, Ryan Culpepper, Matthew Flatt, and Matthias
1088 Felleisen. Languages as libraries. In *PLDI*, pages 132–141, 2011. doi:10.1145/1993498.
1089 1993514.

1090 53 Jose Valim. Elixir v1.18 released, 2024. [https://elixir-lang.org/blog/2024/12/19/](https://elixir-lang.org/blog/2024/12/19/elixir-v1-18-0-released/)
1091 [elixir-v1-18-0-released/](https://elixir-lang.org/blog/2024/12/19/elixir-v1-18-0-released/). Accessed 2025-03-05.

1092 54 Wikipedia. Fallacies of distributed computing, 2024. [https://en.wikipedia.org/wiki/](https://en.wikipedia.org/wiki/Fallacies_of_distributed_computing)
1093 [Fallacies_of_distributed_computing](https://en.wikipedia.org/wiki/Fallacies_of_distributed_computing). Accessed 2025-02-28.

1094 55 Thomas Wu. The secure remote password protocol, 1997. [http://srp.stanford.edu/ndss.](http://srp.stanford.edu/ndss.html)
1095 [html](http://srp.stanford.edu/ndss.html). Accessed 2025-03-04.

